

UML-Pipeline — System Alignment Report

Automated UML Dataset Generation from Natural-Language Requirements with Multimodal Verification for Software Design

Student: **Dipak Yadav** | Advisor: **Yutong Zhao, Ph.D.**
Department of Computer Engineering & Computer Science
California State University, Long Beach
Software artifact: github.com/dipak5501/uml-generation-pipeline
Document purpose: Map the implemented program to the paper methodology

1. Purpose of This Document

This report maps the paper method to the implemented system. It explains how the software system implements the end-to-end methodology described in the thesis paper: generating design-phase UML artifacts from natural-language requirements (and, as an application extension, from source code), rendering them with PlantUML, and verifying quality using a multimodal vision-language model (VLM) ensemble with MMMU-weighted scoring, plus human-evaluation support.

2. Thesis Contributions Mapped to Software

The thesis proposes a three-stage AI pipeline. The application realizes each stage as executable services with persistence, API, and UI.

Thesis element	Paper claim	Implemented in software
Stage 1 — Spec generation	Lightweight LLM converts free-form requirements into structured technical specifications for design-phase modeling.	TechnicalSpecification Generator : requirement → structured spec (architect prompts). Also supports source-code → spec recovery.
Stage 2 — PlantUML synthesis	Reasoning-oriented LLM synthesizes syntactically valid PlantUML for class, object, component, and package diagrams.	PlantUML Generator with diagram-specific versioned prompts under <code>prompts/</code> . Also supports flowchart (activity) diagrams in the application.
Stage 3 — Render + multimodal verification	PlantUML is rendered; diagram images are scored by three VLMs; scores are combined with MMMU weights; invalid renders score 0.	Renderer (local JAR or remote PlantUML fallback) + Validation Service with Qwen / LLaMA-Vision / Aya-Vision weights 53.1 / 50.7 / 39.9 and paper composite formula.
Human evaluation	Expert rubric ratings and correlation with automated scores.	Human Evaluation UI/API: semantic correctness, structural completeness, syntactic accuracy, overall coherence + comments; analytics correlation when reviews exist.
Dataset / scale	Large verified design-phase UML dataset generation and analysis.	Batch jobs, SQLite artifact store, export CSV/JSONL/Parquet, analytics distributions, batch dataset scripts (≥200 artifacts).

3. End-to-End Runtime Pipeline (How It Works)

When a user submits either a natural-language requirement or source code, the system executes the following thesis-aligned flow:

- **Intake** — Validate input; select diagram type (class / object / component / package / flowchart).
- **Specification** — Produce a structured technical specification (entities, relationships, modules/process steps).
- **PlantUML generation** — Convert the specification into diagram-specific PlantUML.

- **Static validation + repair** — Syntax/semantic guards (especially package diagrams); targeted repair retries when needed.
- **Render gate** — Render PNG/SVG. If rendering fails, composite score is forced to **0** (paper rule).
- **Multimodal scoring** — Three VLMs each assign 0–6. Composite is the MMMU-weighted average of all three scores (zeros included), matching thesis Eq. (weighted).
- **Persist + review** — Store full artifact trace; optional human rubric; analytics/export.

4. Scoring Formula (Paper-Faithful)

Let $\text{score}_i \in \{0, \dots, 6\}$ be the score from model i with weight w_i , and let $\delta=1$ when the diagram passes the render gate (else $\delta=0$). Then:

$$S = \delta \cdot \sum (\text{score}_i \times w_i) / \sum w_i$$

$A = 1$ if at least two models score $\geq \tau$ ($\tau=4$); dataset entry if $A=1$ and $S \geq 3$.

Weights implemented exactly as in the thesis configuration: Qwen2.5-VL-3B = **53.1**, LLaMA-3.2-11B-Vision = **50.7**, Aya-Vision-8B = **39.9**.

5. Application Features Beyond Scripts

To make the methodology runnable as software, the research pipeline was wrapped into a full application:

- **FastAPI backend** — generation, jobs, artifacts, rescore/repair, human review, analytics, export.
- **Streamlit UI** — dashboard, free-text/code generation, batch dataset runs, artifact review, human evaluation, analytics, settings/health.
- **SQLite persistence** — full requirement $\rightarrow$ spec $\rightarrow$ PlantUML $\rightarrow$ image $\rightarrow$ model scores $\rightarrow$ composite.
- **Provider abstraction** — OpenAI-compatible, Ollama, and mock mode for offline runs.
- **Docker / deploy docs** — local and cloud launch instructions.

6. Evidence Checklist for Faculty Demo

The following checklist shows what can be observed in a live run to confirm paper–system alignment.

#	Thesis behavior	Where to see it in the program
1	Requirement → technical specification	Generate page → Technical specification panel
2	Spec → PlantUML for design diagrams	Generate page → PlantUML code + download
3	Render gate / failure ⇒ score 0	Paper validation pipeline strip + composite metric
4	Three VLM scores + weighted composite	Per-model VLM scores table + final weighted score
5	Package difficulty / repair awareness	Package diagram type + repair attempts history
6	Human expert-style evaluation	Human Evaluation page (4 rubric dimensions)
7	Dataset-style batch generation & analytics	Batch Generation + Analytics + export endpoints
8	Traceable artifacts for audit	Artifact Review: full stored trace per sample

7. How to Run the Demonstration

Local run (recommended)

```
cd uml-generation-pipeline
make install
make api    # http://127.0.0.1:8000/docs
make ui     # http://127.0.0.1:8501
```

Default `MOCK_PROVIDERS=true` allows a complete offline run without paid APIs. Rendering uses PlantUML (local Java if available, otherwise remote fallback). For live models, set `MOCK_PROVIDERS=false` and configure Ollama or an OpenAI-compatible endpoint.

Suggested 5-minute usage path

- Open the UI home/dashboard and show artifact counts and mean scores.
- Generate from a plain-English requirement (class diagram) and show the full trace.
- Switch to source-code mode, paste sample code, generate a class diagram.
- Point to the validation strip: syntax → render gate → VLM scores → composite.
- Open Human Evaluation and submit a rubric score; show Analytics page.

8. Repository Layout (Implementation Surface)

```
app/          FastAPI services, providers, scoring, repair, orchestration
ui/           Streamlit application UI
uml_pipeline/ Core research pipeline (render, scoring helpers)
prompts/      Versioned prompt templates aligned to thesis stages
paper/        LaTeX thesis paper sources
tests/        Scoring, validators, code analysis, API smoke tests
docs/         Plans, assumptions, deploy guide, this report
```

9. Implementation Notes (Scope Boundaries)

- The software implements the **methodology** of the thesis (pipeline stages, diagram types, render gate, weighted VLM ensemble, human rubric, analytics).

- A full 8,000-sample paper-scale experiment still depends on provisioned LLM/VLM compute; the app supports that mode and also an offline mock mode for reliable local execution.
- Flowchart diagrams and source-code intake are **application extensions** that reuse the same specification → PlantUML → multimodal validation pipeline.
- Private model reasoning is not exposed in UI/logs; only final specs, PlantUML, validation, and scores are persisted.

10. Conclusion

The implemented system is not merely a collection of scripts: it is a runnable implementation of *Automated UML Dataset Generation from Natural-Language Requirements with Multimodal Verification for Software Design*. It operationalizes dual-stage generation, PlantUML rendering as a hard quality gate, MMMU-weighted multimodal verification, human evaluation, and dataset analytics—matching the paper’s methodological core while providing an interface for inspection and evaluation.

Contact: Dipak.yadav5501@gmail.com · Repository: <https://github.com/dipak5501/uml-generation-pipeline>